การออกแบบสถาปัตยกรรมของระบบ

System Architecture Design

Version 1.0

ระบบจัดซื้อจัดจ้างขนาดย่อ (Mini e-Procurement)

จัดทำโดย บารเมษฐ์ นามมุงคุณ

ชื่อโครงการ	ระบบจัดซื้อจัดจ้างขนาดย่อ (Mini e-Procurement)
เลขที่สัญญา	ไม่มี (โจทย์สมมติเพื่อแสดงผลงาน)
ชื่อเอกสาร	การออกแบบสถาปัตยกรรมของระบบ (System Architecture Design)
วันที่เอกสาร	4 พฤศจิกายน 2569
Version เอกสาร	1.0

จัดทำโดย		
ชื่อ-นามสกุล	**ตำแหน่ง**	**วันที่**
บารเมษฐ์ นามมุงคุณ	System Analyst	4 พฤศจิกายน 2569

ตรวจสอบโดย		
ชื่อ-นามสกุล	**ตำแหน่ง**	**วันที่**
ยังไม่มี (เอกสารตัวอย่าง)	-	-

อนุมัติโดย		
ชื่อ-นามสกุล	**ตำแหน่ง**	**วันที่**
ยังไม่มี (เอกสารตัวอย่าง)	-	-

ประวัติการแก้ไข

Version	วันที่	ผู้แก้ไข	รายละเอียด
1.0	4 พฤศจิกายน 2569	บารเมษฐ์ นามมุงคุณ	จัดทำฉบับแรก (ร่างโดย AI รอผู้จัดทำตรวจรับ)

สารบัญ

1. บทนำและขอบเขต **1**

2. ภาพรวมสถาปัตยกรรม **1**

2.1 บริบทของระบบ 1

2.2 แผนภาพหน่วยประกอบ 2

2.3 รายละเอียดหน่วยประกอบ 2

3. การแบ่งโมดูลของ API **3**

4. มุมมองขณะทำงาน (Runtime View) **4**

5. โครงสร้างโค้ดของ API service (Development View) **17**

6. การติดตั้ง **20**

6.1 แผนภาพการติดตั้ง 20

6.2 สภาพแวดล้อม 20

7. ความมั่นคงปลอดภัย **21**

7.1 ขอบเขตความเชื่อถือ 21

7.2 มาตรการหลัก 21

7.3 การเทียบกับ OWASP Top 10 22

8. คุณลักษณะคุณภาพและมาตรการเชิงสถาปัตยกรรม **22**

9. การตัดสินใจเชิงสถาปัตยกรรม **23**

สารบัญรูปภาพ

รูปที่ 2.1-1 บริบทของระบบ 1
รูปที่ 2.2-1 หน่วยประกอบของระบบ 2
รูปที่ 4-1 ลำดับตรวจ token บทบาท และขอบเขตข้อมูลก่อนเข้า operation 4
รูปที่ 4-2 ลำดับการอ่านรายการใบขอซื้อ รายละเอียด และกล่องอนุมัติ 5
รูปที่ 4-3 ลำดับการสร้างและแก้ใบขอซื้อฉบับร่าง 6
รูปที่ 4-4 ลำดับการทำงานขณะส่งใบขอซื้อ 8
รูปที่ 4-5 ลำดับการทำงานขณะอนุมัติขั้นสุดท้าย 9
รูปที่ 4-6 ลำดับการสร้าง RFQ อ่านรายละเอียด และบันทึกใบเสนอราคา 10
รูปที่ 4-7 ลำดับการเทียบราคาและเสนอผลเลือกผู้ชนะ 12
รูปที่ 4-8 ลำดับการสร้าง อ่าน และออกใบสั่งซื้อ 13
รูปที่ 4-9 ลำดับการบันทึกรับของ 14
รูปที่ 4-10 ลำดับการจัดการทะเบียนผู้ขาย 15
รูปที่ 4-11 ลำดับการสร้างรายงานสรุปการจัดซื้อและการอ่าน audit trail 16
รูปที่ 6.1-1 มุมมองการติดตั้งในสภาพแวดล้อมใช้งานจริง 20
รูปที่ 7.1-1 ขอบเขตความเชื่อถือ 21

สารบัญตาราง

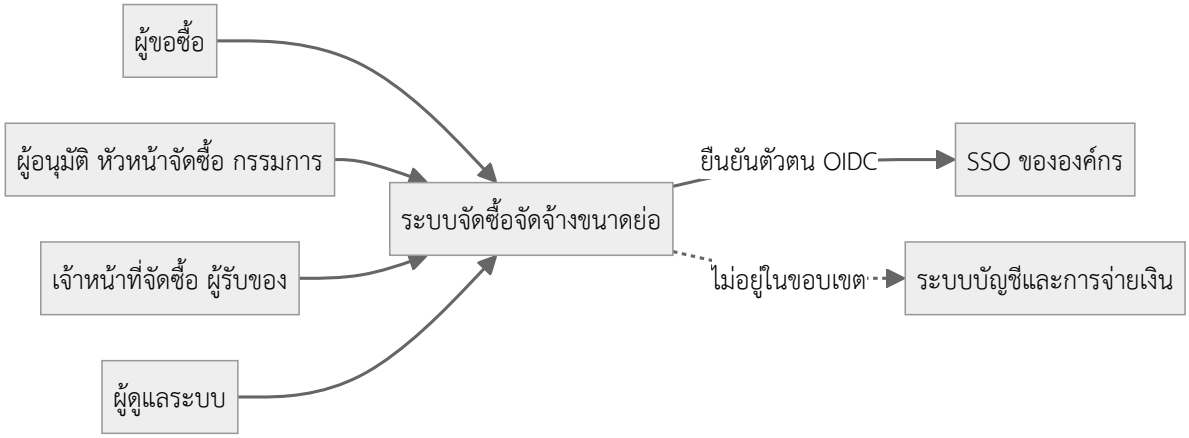
ตารางที่ 2.3-1 หน่วยประกอบของระบบ 3
ตารางที่ 3-1 โมดูลของ API service 3
ตารางที่ 4-1 ผู้ร่วมใน runtime ของการตรวจสิทธิ์ 4
ตารางที่ 4-2 ลำดับและผลของการตรวจสิทธิ์ 5
ตารางที่ 4-3 ลำดับการทำงานของการอ่านใบขอซื้อ 6
ตารางที่ 4-4 ลำดับการทำงานของการสร้างและแก้ร่าง 7
ตารางที่ 4-5 ลำดับและผลของการส่งใบขอซื้อ 9
ตารางที่ 4-6 ลำดับและผลของการตัดสินการอนุมัติ 10
ตารางที่ 4-7 ลำดับการทำงานของ RFQ และใบเสนอราคา 11
ตารางที่ 4-8 ลำดับการทำงานของการเทียบราคาและเสนอผล 13
ตารางที่ 4-9 ลำดับการทำงานของใบสั่งซื้อ 14
ตารางที่ 4-10 ลำดับการทำงานของการรับของ 15
ตารางที่ 4-11 ลำดับการทำงานของทะเบียนผู้ขาย 16
ตารางที่ 4-12 สูตรและลำดับการทำงานของรายงานและ audit trail 17
ตารางที่ 5.1-1 ชั้นของโค้ดในแต่ละโมดูล 17
ตารางที่ 5.1-2 ทิศทางการพึ่งพาของโมดูล 18
ตารางที่ 5.2-1 โมดูลของ API service กับตารางที่เป็นเจ้าของ 18
ตารางที่ 5.3-1 ส่วนประกอบของชุดรันบนเครื่องนักพัฒนา 19
ตารางที่ 5.3-2 ข้อจำกัดของ stack ที่ต้องรู้ก่อนเขียนโค้ด 19
ตารางที่ 6.2-1 สภาพแวดล้อมของระบบ 21
ตารางที่ 7.3-1 ความเสี่ยงตาม OWASP และมาตรการ 22
ตารางที่ 8-1 คุณลักษณะคุณภาพและมาตรการ 22

1. บทนำและขอบเขต

เอกสารนี้อธิบายสถาปัตยกรรมของระบบจัดซื้อจัดจ้างขนาดย่อ: บริบท หน่วยประกอบ การแบ่งโมดูลของ API มุมมองขณะทำงาน การติดตั้ง ความมั่นคงปลอดภัย ความสัมพันธ์กับคุณลักษณะคุณภาพ และการตัดสินใจสำคัญ ทุกข้อเสนอเป็นร่างของ AI จากข้อกำหนดในเอกสารอื่น ผู้จัดทำต้องตรวจรับก่อนใช้ เทคโนโลยีที่เลือกสอดคล้องกับ schema ที่ออกแบบไว้ (SQL Server) ไม่ได้ผ่านการทดลองใช้งาน

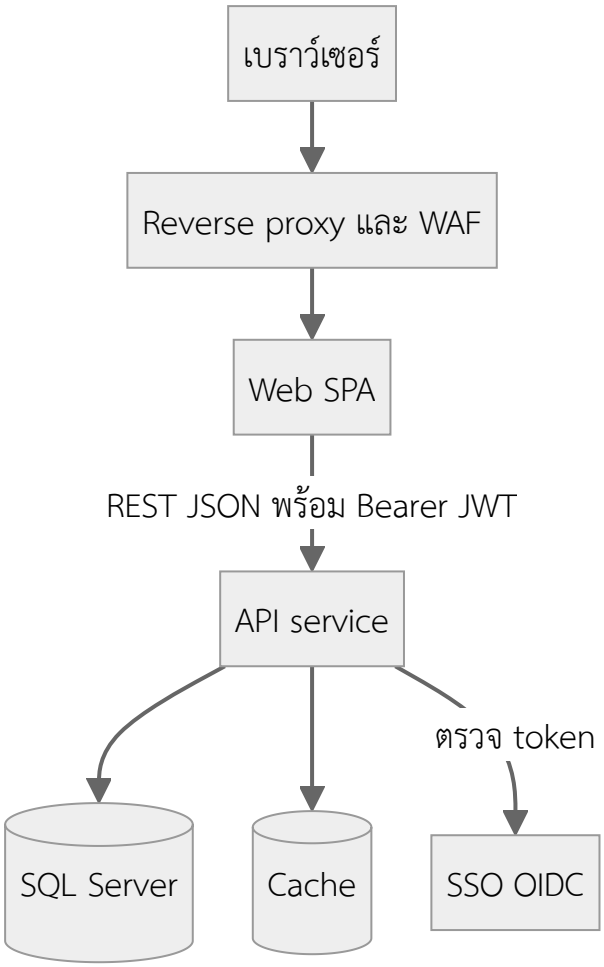
2. ภาพรวมสถาปัตยกรรม

2.1 บริบทของระบบ



รูปที่ 2.1-1 บริบทของระบบ

2.2 แผนภาพหน่วยประกอบ



รูปที่ 2.2-1 หน่วยประกอบของระบบ

2.3 รายละเอียดหน่วยประกอบ

หน่วย	เทคโนโลยี	หน้าที่	เจ้าของ
Reverse proxy และ WAF	TLS, ตัวกรองคำขอ, จำกัดอัตราการเรียก	ปลายทางเดียวที่เปิดสู่ผู้ใช้ ยกเลิก TLS และตัดคำขอผิดปกติก่อนถึง API	ภายใน (ผู้ให้บริการโครงสร้างพื้นฐาน)
Web SPA	React และ TypeScript	แสดง 10 หน้าจอ จัดรูปแบบวันที่ พ.ศ. และเงินบาท สร้าง Idempotency-Key ต่อการกระทำ และจัดการข้อผิดพลาดตามรหัส	ภายใน
API service	ASP.NET Core (.NET 8) แบบ modular monolith	บังคับสิทธิ์ ขอบเขตข้อมูล กฎธุรกิจ และขอบเขตธุรกรรมของทุก operation	ภายใน

หน่วย	เทคโนโลยี	หน้าที่	เจ้าของ
ฐานข้อมูล	SQL Server 2022	ที่เก็บข้อมูลหลัก ตาม `sql/schema.sql` รวมขั้นตอนออกเลขเอกสารและตาราง audit แบบเพิ่มอย่างเดียว	ภายใน
แคช	Redis	เก็บผลของคำสั่งที่มี Idempotency-Key 24 ชั่วโมง ไม่ใช่แหล่งข้อมูลหลัก	ภายใน
SSO	OpenID Connect ขององค์กร	ยืนยันตัวตนและออก JWT ที่มีรหัสผู้ใช้และบทบาท	ภายนอก

ตารางที่ 2.3-1 หน่วยประกอบของระบบ

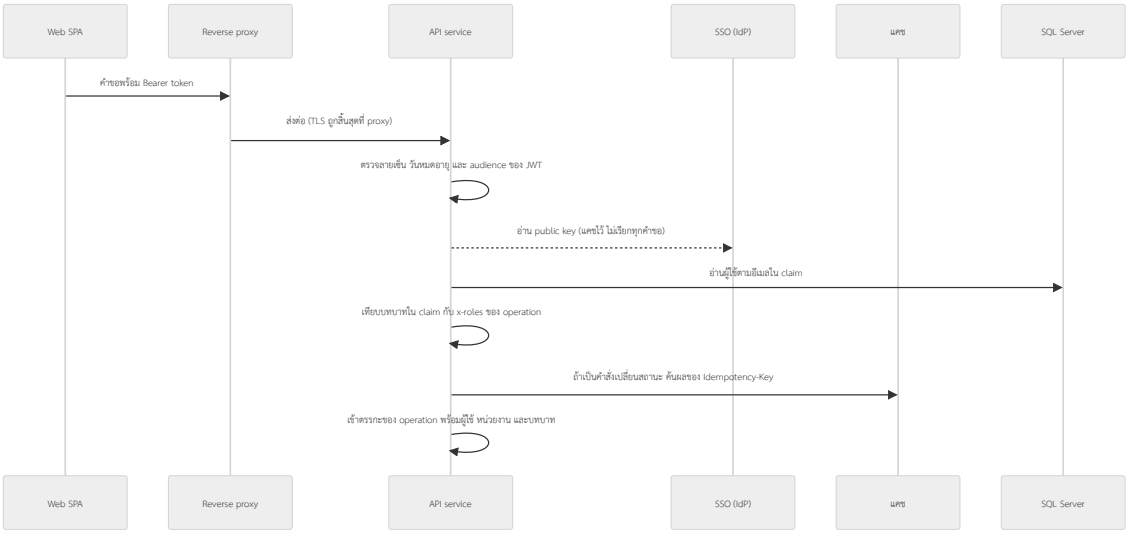
3. การแบ่งโมดูลของ API

โมดูล	หน้าที่	FR	API operation
โมดูลใบขอซื้อ	สร้าง แก้ ค้นหา และส่ง PR คำนวณวงเงินรวม	FR-01, FR-02, FR-03	listPurchaseRequests, createPurchaseRequest, getPurchaseRequest, updatePurchaseRequest, submitPurchaseRequest
โมดูลอนุมัติ	สร้างสายอนุมัติตามวงเงิน ตรวจการแยกหน้าที่ และเปลี่ยนสถานะเมื่ออนุมัติขั้นสุดท้าย	FR-04	listMyApprovals, decideApproval
โมดูลจัดหา	สร้าง RFQ รับใบเสนอราคา เปรียบเทียบ และเสนอผลเลือก	FR-05, FR-06, FR-07, FR-08	createRfq, getRfq, recordQuotation, getQuotationComparison, awardRfq
โมดูลสั่งซื้อและรับของ	ออก PO และบันทึกการรับของ คำนวณสถานะ PO และ PR	FR-09, FR-10	createPurchaseOrder, getPurchaseOrder, issuePurchaseOrder, recordGoodsReceipt
โมดูลผู้ขาย	ทะเบียนผู้ขายและการระงับ	FR-11	listVendors, createVendor, setVendorStatus
โมดูลรายงาน	รายงานสรุปการจัดซื้อแบบอ่านอย่างเดียว	FR-12	getSpendSummary
โมดูล audit	บันทึกและอ่านเหตุการณ์ ทุกโมดูลเรียกใช้ตัวเขียน	FR-13	listAuditLogs
โมดูลแพลตฟอร์ม	ใช้ร่วมกันทุกโมดูล: ตรวจสิทธิ์ ออกเลขเอกสาร Idempotency-Key รูปแบบข้อผิดพลาด และตรวจการแก้ไขซ้อน	FR-14	

ตารางที่ 3-1 โมดูลของ API service

4. มุมมองขณะทำงาน (Runtime View)

ทุก operation ผ่านลำดับนี้ก่อนถึงตรรกะของตนเอง ตรวจเรียงจากถูกไปแพง และหยุดที่ขั้นแรกที่ไม่ผ่าน



รูปที่ 4-1 ลำดับตรวจ token บทบาท และขอบเขตข้อมูลก่อนเข้า operation

ผู้ร่วม	หน้าที่ในลำดับนี้
Reverse proxy	สิ้นสุด TLS ส่งหัว `Authorization` ต่อให้ API ไม่ตรวจเนื้อหา token
API service	ตรวจ token อ่านผู้ใช้ ตรวจบทบาทและขอบเขต ตัดสินว่าเข้าตรรกะได้หรือไม่
SSO (IdP)	ออก JWT และเผยแพร่ public key; ไม่ถูกเรียกทุกคำขอ
แคช	เก็บผลของ `Idempotency-Key` 24 ชั่วโมง
SQL Server	เก็บผู้ใช้ (`app_user`) ใช้ตรวจสถานะใช้งานและหน่วยงาน

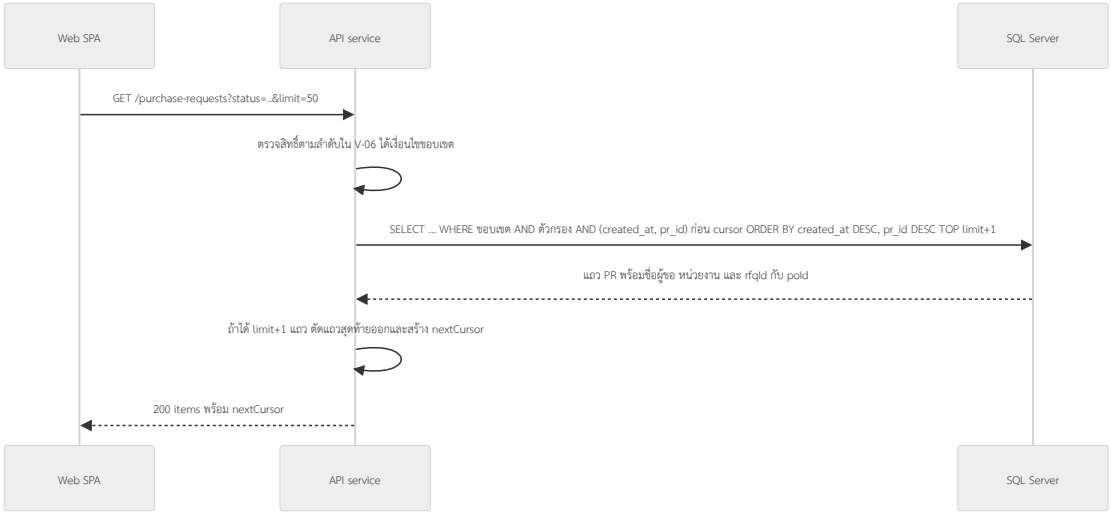
ตารางที่ 4-1 ผู้ร่วมใน runtime ของการตรวจสิทธิ์

ลำดับ	ตรวจอะไร	ไม่ผ่านแล้ว	หมายเหตุ
1	มี `Authorization: Bearer` และ JWT ลายเซ็น วันหมดอายุ audience ถูกต้อง	`401` `ERR-AUTH-001`	ใช้ public key ที่แคชไว้ ถ้าไม่พบ `kid` จึงอ่านใหม่หนึ่งครั้ง
2	ผู้ใช้ตามอีเมลใน claim มีอยู่ใน `app_user` และ `is_active` เป็นจริง	`403` `ERR-AUTH-002`	ผู้ใช้ที่ SSO รู้จักแต่ยังไม่ถูกเพิ่มในระบบ เข้าใช้ไม่ได้
3	บทบาทใน claim อยู่ใน `x-roles` ของ operation	`403` `ERR-AUTH-002`	บทบาทมาจาก claim ไม่รับจาก body
4	ขอบเขตข้อมูล: ผู้ขอซื้อเห็นเฉพาะ PR ของตน ผู้อนุมัติหน่วยงานเห็น PR ของหน่วยงานตน บทบาทอื่นที่เรียก operation นั้นได้เห็นทุกหน่วยงาน	PR ที่อยู่นอกขอบเขตตอบ `404` `ERR-NF-001` (ไม่บอกว่ามีอยู่) ทั้งการอ่านและการสั่งแก้หรือส่ง	ใช้เงื่อนไขในคำสั่งอ่านฐานข้อมูล ไม่กรองหลังอ่านใน memory

ลำดับ	ตรวจอะไร	ไม่ผ่านแล้ว	หมายเหตุ
5	คำสั่งเปลี่ยนสถานะ: `Idempotency-Key` ตรงกับที่เคยเห็นหรือไม่	คีย์เดิม payload ต่าง `422 ERR-IDEM-001`; คีย์เดิม payload เดิมตอบผลเดิมโดยไม่ทำซ้ำ	ใช้ร่วมทุก operation ที่รับคีย์
6	ตรวจ BR-04 (แยกหน้าที่) เฉพาะ operation อนุมัติ	`403 ERR-SOD-001`	ตรวจใน service ของการอนุมัติ ดู V-04

ตารางที่ 4-2 ลำดับและผลของการตรวจสิทธิ์

ผู้ใช้และบทบาทที่ผ่านการตรวจถูกส่งต่อให้ตรรกะของ operation เป็นค่าเดียว ทุก operation อ่านผู้ใช้จากค่านี้ ไม่อ่านซ้ำจาก token

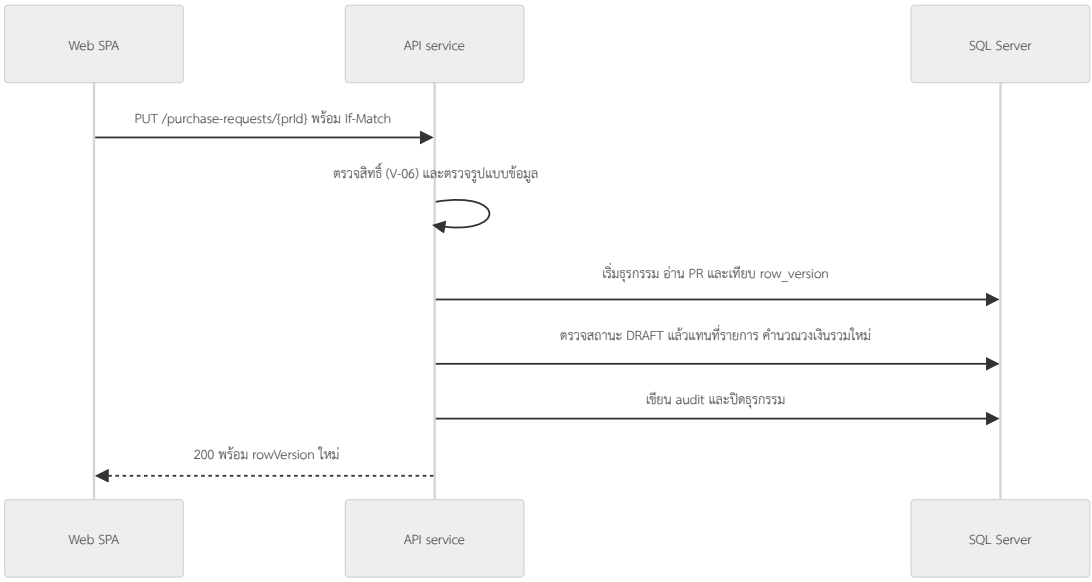


รูปที่ 4-2 ลำดับการอ่านรายการใบขอซื้อ รายละเอียด และกล่องอนุมัติ

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
1	`listPurchaseRequests`	ใส่เงื่อนไขขอบเขตตามบทบาทในคำสั่ง SQL แล้วกรองด้วย `status`, `createdFrom`, `createdTo`, `requesterId`	แถวเรียงใหม่สุดก่อน ใช้ดัชนี `ix_pr_status_created` และ `ix_pr_requester` (NFR-01)
2	`listPurchaseRequests`	อ่านเกินหนึ่งแถว (`limit` + 1) เพื่อรู้ว่ามีหน้าถัดไป `nextCursor` คือ (`created_at`, `pr_id`) ของแถวสุดท้ายที่ส่งออก เข้ารหัส base64	`nextCursor` เป็น null เมื่อไม่มีหน้าถัดไป
3	`listPurchaseRequests`	`rfqId` คือ RFQ ล่าสุดของ PR และ `poId` คือ PO ที่ไม่ถูกยกเลิกของ PR	ใช้นำทางจากหน้ารายการ ค่าว่างเมื่อยังไม่มี

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
4	`getPurchaseRequest`	อ่าน PR พร้อมรายการ (`pr_item` เรียงตาม `line_no`) และแถวอนุมัติของ PR (`approval` ที่ `doc_type` เป็น `PR` เรียงตาม `step_no`) ภายใต้เงื่อนไขขอบเขตเดียวกัน	`404 ERR-NF-001` เมื่อไม่พบหรืออยู่นอกขอบเขต; `rowVersion` ส่งเป็นสตริงฐานสิบหก
5	`listMyApprovals`	อ่านแถว `approval` ที่ `decision` เป็น `PENDING` และ `step_no` เป็นขั้นต่ำสุดที่ยังรออยู่ของเอกสาร และ `required_role` ตรงบทบาทผู้เรียก ใช้ดัชนี `ix_approval_pending` แล้วเติมเลขที่ หัวข้อ ผู้ขอ วงเงิน จากเอกสารต้นทางตาม `doc_type`	เรียงเก่าสุดก่อน; `proposerId` คือผู้ขอซื้อ (PR) หรือผู้เสนอผล (AWARD) เพื่อให้หน้าซ่อนปุ่มตาม BR-04

ตารางที่ 4-3 ลำดับการทำงานของการอ่านใบขอซื้อ

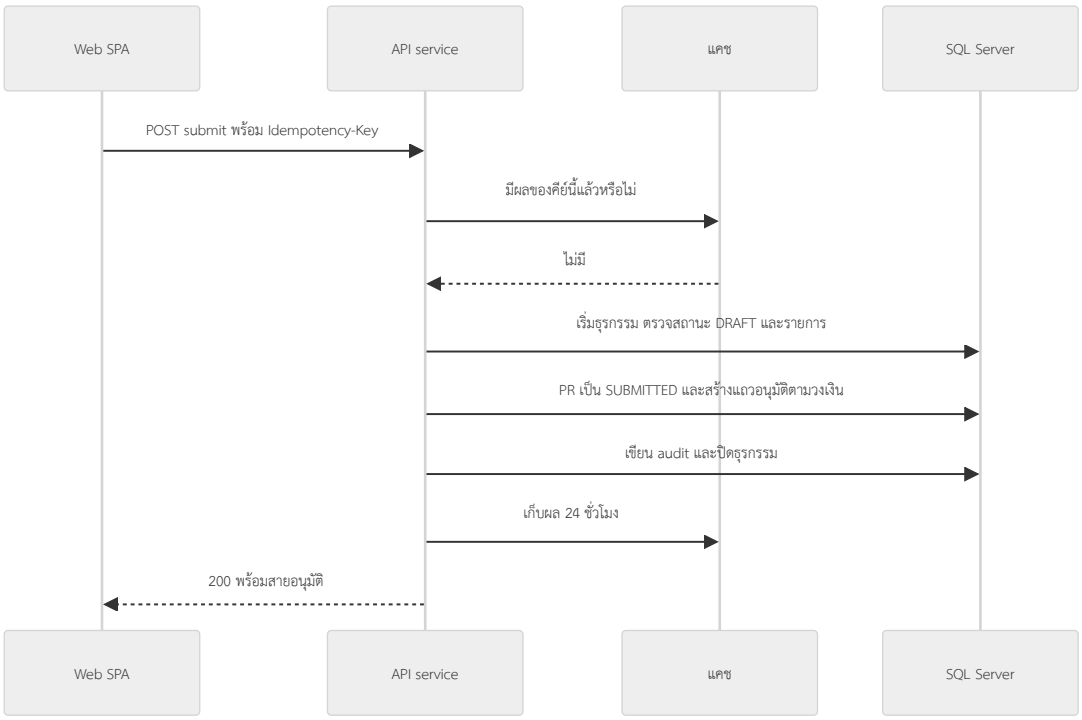


รูปที่ 4-3 ลำดับการสร้างและแก้ใบขอซื้อฉบับร่าง

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
1	ทั้งคู่	ตรวจรูปแบบตาม schema: รายการ 1–100 แถว จำนวนมากกว่า 0 ทศนิยมไม่เกิน 3 ตำแหน่ง ราคาไม่ติดลบทศนิยมไม่เกิน 2 ตำแหน่ง วงเงินรวมไม่เกินช่วงของคอลัมน์ วันที่ต้องการใช้ไม่ก่อนวันนี้ (ปฏิทิน Asia/Bangkok)	`422 ERR-VAL-001` พร้อม `details[]` ต่อฟิลด์

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
2	`createPurchaseRequest`	เรียก `usp_next_doc_no('PR')` ก่อนเริ่มธุรกรรมธุรกิจ (ด้วยการเชื่อมต่อและธุรกรรมของตัวเอง) ได้เลขที่ `PR-YYYY-NNNNN` (NFR-04) แล้วจึงเริ่มธุรกรรม insert `purchase_request` สถานะ `DRAFT` และ `pr_item` เรียง `line_no` ตั้งแต่ 1; ถ้าเรียกข้างในธุรกรรมธุรกิจ การย้อนธุรกรรมจะย้อนเลขที่ด้วย	`201` พร้อม `rowVersion`; เลขที่ที่ออกแล้วไม่ถูกใช้ซ้ำแม้ธุรกรรมภายหลังล้ม (เลขอาจข้าม)
3	`updatePurchaseRequest`	อ่าน PR ล็อกแถว เทียบ `row_version` กับ `If-Match`	ไม่ตรง `409 ERR-CONC-001`
4	`updatePurchaseRequest`	สถานะต้องเป็น `DRAFT` เท่านั้น เอกสารที่ส่งแล้วแก้ไม่ได้ (BR-12)	สถานะอื่น `409 ERR-STATE-001`
5	`updatePurchaseRequest`	แทนที่รายการทั้งชุด (ลบแล้วใส่ใหม่เรียง `line_no` ใหม่)	ผู้ใช้ส่งรายการครบทั้งชุดทุกครั้ง
6	ทั้งคู่	คำนวณวงเงินรวม: ราคาบรรทัด = ปัดเศษ 2 ตำแหน่งของ (จำนวน × ราคาต่อหน่วยประมาณการ) วงเงินรวม = ผลรวมของราคาบรรทัด เก็บใน `total_estimated`	เซิร์ฟเวอร์คำนวณเสมอ ไม่รับค่านี้จาก client
7	ทั้งคู่	เขียน `audit_log` (`CREATE` หรือ `UPDATE`) ในธุรกรรมเดียวกัน แล้วอ่านแถวกลับเพื่อส่ง `rowVersion` ใหม่	ผลที่ส่งกลับตรงกับที่บันทึกจริง

ตารางที่ 4-4 ลำดับการทำงานของการสร้างและแก้ร่าง

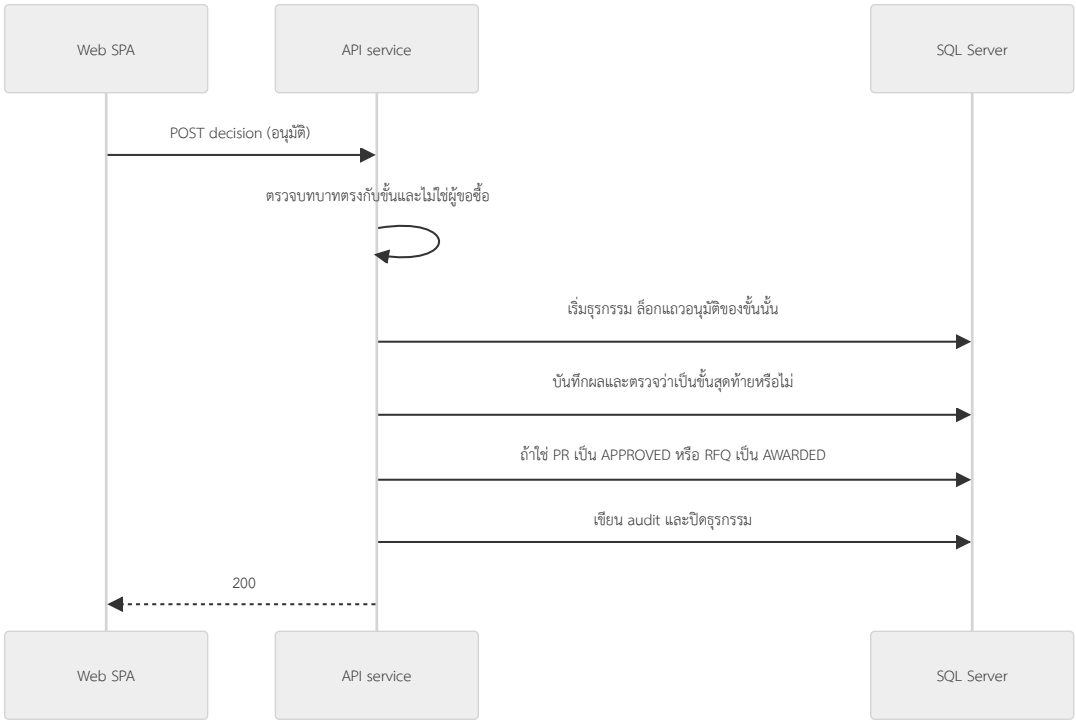


รูปที่ 4-4 ลำดับการทำงานขณะส่งใบขอซื้อ

ลำดับ	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
1	ค้นแคชด้วย `Idempotency-Key` ของผู้เรียก	คีย์เดิม payload เดิมตอบผลเดิมทันที โดยไม่ทำซ้ำ; ไม่มีหัวนี้ `422` `ERR-VAL-001`; คีย์เดิม payload ต่าง `422` `ERR-IDEM-001`
2	เริ่มธุรกรรม อ่าน PR และล็อกแถว PR ต้องเป็นของผู้เรียก	ไม่ใช่ของตนหรือไม่พบ `404` `ERR-NF-001`
3	สถานะต้องเป็น `DRAFT` (BR-12) และมีรายการอย่างน้อยหนึ่งรายการ	สถานะอื่น `409` `ERR-STATE-001`
4	สร้างแถว `approval` ชนิด `PR` ตามวงเงินรวม (BR-03): ไม่เกิน 20,000.00 ขั้น `DEPT_APPROVER`; เกิน 20,000.00 ถึง 200,000.00 เพิ่ม `PROCUREMENT_HEAD`; เกิน 200,000.00 เพิ่ม `COMMITTEE` เรียง `step_no` ตั้งแต่ 1 ทุกแถว `PENDING`	วงเงินที่ขอบช่วงอยู่ในช่วงล่าง (20,000.00 ได้ 1 ขั้น 20,000.01 ได้ 2 ขั้น)
5	ตั้ง PR เป็น `SUBMITTED` บันทึก `submitted_at` และ `row_version` ใหม่ เขียน `audit_log` (`SUBMIT`) แล้วปิดธุรกรรม	

ลำดับ	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
6	เก็บผลในแคช 24 ชั่วโมง ตอบ `200` พร้อมสายอนุมัติ	แคชถูกล้างแล้วส่งซ้ำด้วยคีย์เดิม: ด่านสถานะในลำดับที่ 3 กันการสร้างซ้ำ ตอบ `409` `ERR-STATE-001` และไม่สร้างแถวอนุมัติหรือ audit เพิ่ม

ตารางที่ 4-5 ลำดับและผลของการส่งใบขอซื้อ

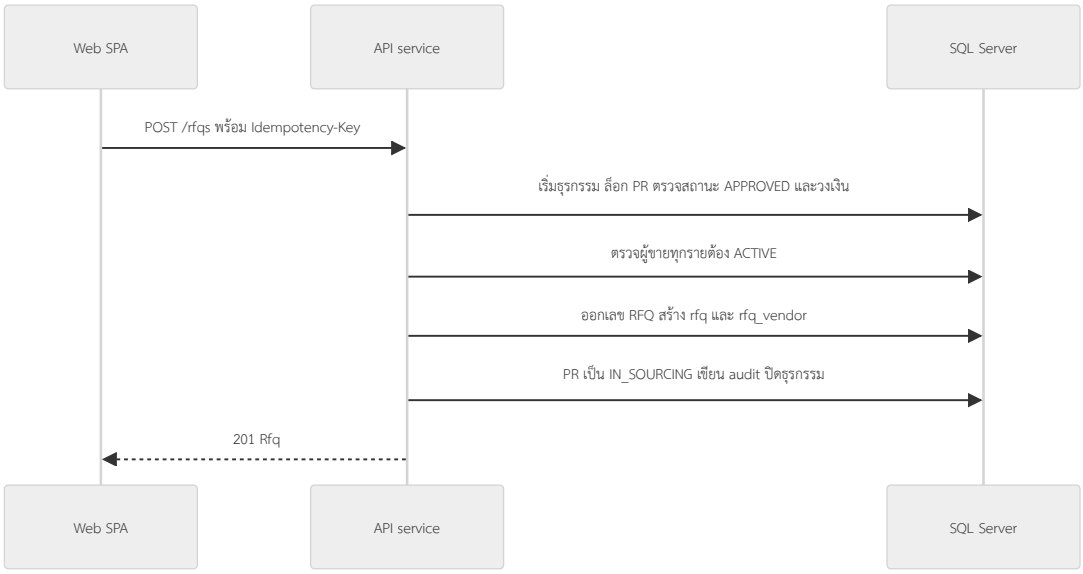


รูปที่ 4-5 ลำดับการทำงานขณะอนุมัติขั้นสุดท้าย

ลำดับ	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
1	หาแถว `approval` ตาม `approvalId`; ผู้อนุมัติหน่วยงานเห็นเฉพาะแถวของ PR ในหน่วยงานตน บทบาทอื่นเห็นทุกหน่วยงาน	ไม่พบหรืออยู่นอกขอบเขต `404` `ERR-NF-001`
2	บทบาทใน claim ต้องมี `required_role` ของแถวนี้	ไม่มี `403` `ERR-AUTH-002`
3	แยกหน้าที่ (BR-04): ผู้เรียกต้องไม่ใช่ผู้ขอซื้อของ PR (ชนิด `PR`) หรือผู้เสนอผล `awarded_by` (ชนิด `AWARD`) เทียบรหัสผู้ใช้	`403` `ERR-SOD-001`
4	ตรวจ body: `decision` เป็น `APPROVED` หรือ `REJECTED`; `REJECTED` ต้องมี `comment`	`422` `ERR-VAL-001`
5	เริ่มธุรกรรม ล็อกแถวอนุมัติ แถวต้อง `PENDING` และเป็นขั้นต่ำสุดที่ยังรออยู่ของเอกสาร (ตัดสินข้ามขั้นไม่ได้)	ไม่ตรง `409` `ERR-STATE-001`

ลำดับ	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
6	บันทึก `decision` `comment` `decided_at` และเขียน `audit_log` (`APPROVE` หรือ `REJECT`; `entity` เป็น `PR` หรือ `RFQ`)	
7	ถ้าปฏิเสธ: ลบแถว `approval` ทั้งชุดของเอกสารนั้น PR กลับ `DRAFT` (แก้ไขได้ และส่งใหม่จะสร้างสายอนุมัติใหม่) หรือ RFQ กลับ `OPEN` พร้อมลบแถว `award`; เหตุผลอยู่ใน `audit_log`	ประวัติอยู่ที่ `audit_log` ไม่เก็บแถวอนุมัติเก่า
8	ถ้าอนุมัติและไม่มีแถว `PENDING` เหลือ: PR เป็น `APPROVED` หรือ RFQ เป็น `AWARDED` (BR-05)	`200` พร้อมแถวที่ตัดสิน

ตารางที่ 4-6 ลำดับและผลของการตัดสินการอนุมัติ

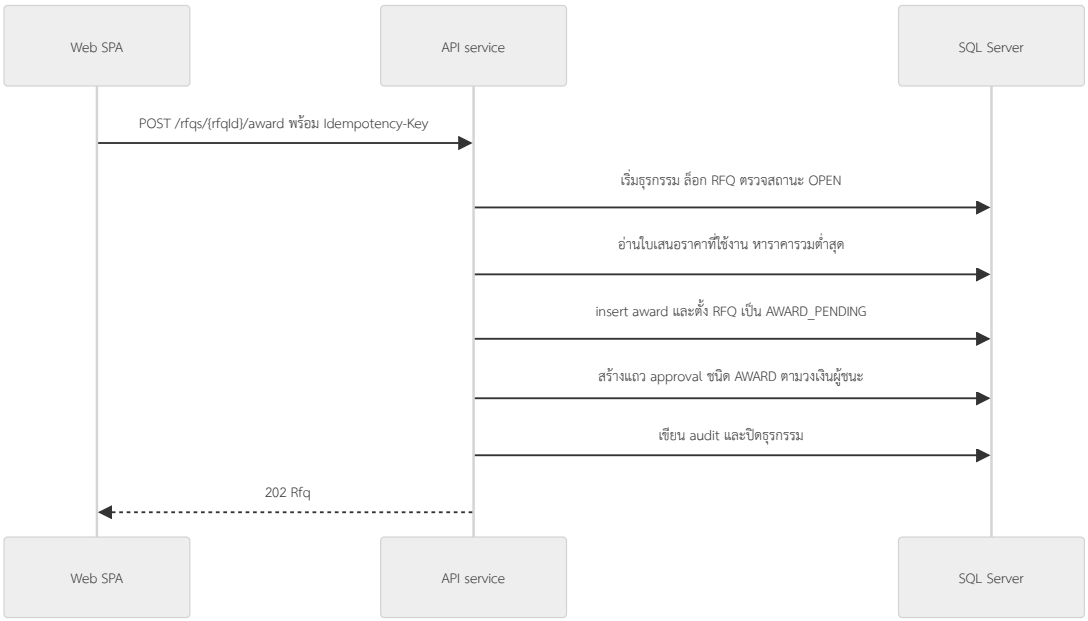


รูปที่ 4-6 ลำดับการสร้าง RFQ อ่านรายละเอียด และบันทึกใบเสนอราคา

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
1	`createRfq`	PR ต้องมีอยู่และเป็น `APPROVED`	ไม่พบ `422` `ERR-VAL-001` ที่ `prId`; สถานะอื่น `409` `ERR-STATE-002`
2	`createRfq`	วงเงินรวมของ PR ต้องเกิน 5,000 บาท (BR-01 ใช้ซื้อตรงแทน)	ไม่เกิน `422` `ERR-RFQ-006`
3	`createRfq`	`vendorIds` ไม่ซ้ำและอย่างน้อย 3 ราย (BR-02) และ `dueAt` ต้องอยู่ในอนาคต	น้อยกว่า 3 `422` `ERR-RFQ-001`; วันที่ผิด `422` `ERR-VAL-001`

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
4	`createRfq`	ผู้ขายทุกรายต้องมีอยู่และ `ACTIVE` (BR-11)	`422 ERR-VND-002`
5	`createRfq`	ออกเลข `RFQ-YYYY-NNNNN` insert `rfq` สถานะ `OPEN` และ `rfq_vendor` ย้าย PR เป็น `IN_SOURCING` เขียน audit	`201`
6	`getRfq`	อ่าน RFQ พร้อมเลขที่และหัวข้อของ PR รายการของ PR และผู้ขายที่เชิญ	`404 ERR-NF-001` เมื่อไม่พบ
7	`recordQuotation`	RFQ ต้องเป็น `OPEN` และเวลาปัจจุบัน (UTC) ไม่เกิน `due_at`	`409 ERR-RFQ-003`
8	`recordQuotation`	ผู้ขายต้องอยู่ใน `rfq_vendor` ของ RFQ นี้ (BR-07)	`422 ERR-RFQ-002`
9	`recordQuotation`	`items` ต้องครอบคลุมทุกรายการของ PR ไม่ซ้ำ ราคารวม = ผลรวมของ ปัดเศษ 2 ตำแหน่ง (จำนวนของ PR × ราคาต่อหน่วยที่เสนอ)	ไม่ครบ `422 ERR-VAL-001`
10	`recordQuotation`	ในธุรกรรมเดียว ตั้ง `is_active` เป็น 0 ให้ฉบับเดิมของผู้ขายรายนี้ (BR-08 เก็บประวัติ) แล้ว insert ฉบับใหม่ `is_active` เป็น 1 ดัชนีเฉพาะ `ux_quotation_active` ป้องกันสองฉบับพร้อมกัน ผู้เรียกที่แพ้การแข่งกันได้ `409 ERR-STATE-001` และส่งใหม่ได้	`201`

ตารางที่ 4-7 ลำดับการทำงานของ RFQ และใบเสนอราคา

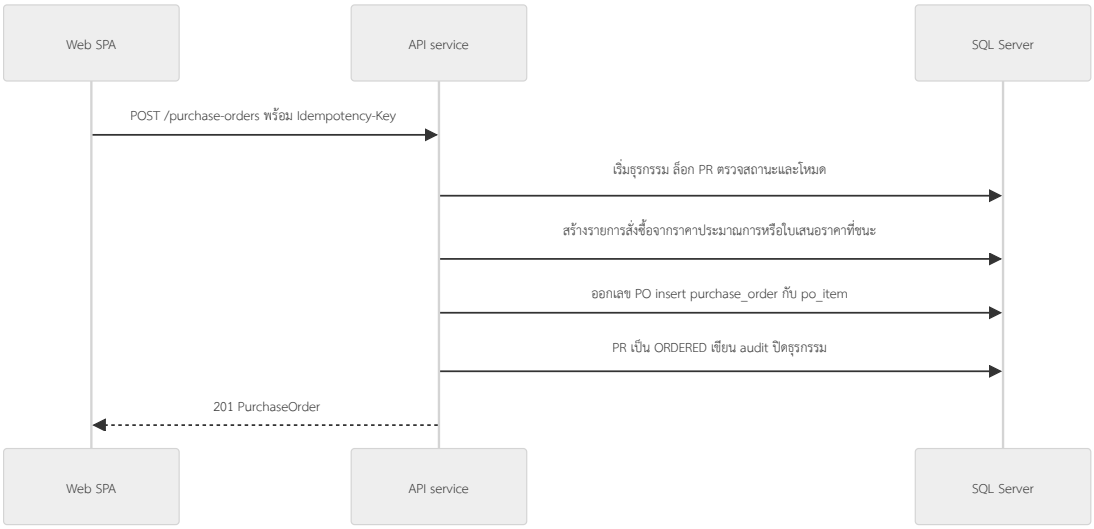


รูปที่ 4-7 ลำดับการเทียบราคาและเสนอผลเลือกผู้ชนะ

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
1	`getQuotationComparison`	อ่านใบเสนอราคาที่ `is_active` เป็น 1 ของ RFQ พร้อมราคาต่อหน่วยต่อรายการและชื่อผู้ขาย เรียงตาม `quotation_id`	`404 ERR-NF-001` เมื่อไม่พบ RFQ
2	`getQuotationComparison`	`lowestQuotationId` คือใบเสนอราคาที่ราคารวมต่ำสุด ถ้าเท่ากันเลือก `quotation_id` ที่น้อยกว่า; เป็น null เมื่อยังไม่มีใบเสนอราคา	ใช้ค่านี้ตัดสินกฎเหตุผลในลำดับที่ 5
3	`awardRfq`	RFQ ต้องเป็น `OPEN`	อื่น `409 ERR-STATE-001`
4	`awardRfq`	ต้องมีใบเสนอราคาที่ใช้งานอย่างน้อย 2 ราย (BR-02) และ `quotationId` ต้องเป็นหนึ่งในนั้น	`422 ERR-RFQ-005`
5	`awardRfq`	ถ้า `quotationId` ไม่ใช่ `lowestQuotationId` ต้องมี `reason` อย่างน้อย 20 ตัวอักษร (BR-06)	`422 ERR-RFQ-004`
6	`awardRfq`	insert `award` ตั้ง RFQ เป็น `AWARD_PENDING`	ผู้เสนอผล (`awarded_by`) คือผู้เรียก
7	`awardRfq`	สร้างแถว `approval` ชนิด `AWARD` (`doc_id` คือ `rfq_id` ของ RFQ นี้): ราคารวมผู้ชนะเกิน 200,000 บาทขั้นเดียว `COMMITTEE` นอกนั้นขั้นเดียว `PROCUREMENT_HEAD` (BR-05)	ผู้เสนอผลอนุมัติเองไม่ได้ (BR-04 ตรวจตอนตัดสิน ดู V-04)

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
8	`awardRfq`	เขียน `audit_log` (`AWARD`) แล้วตอบ `202`	เมื่ออนุมัติครบ RFQ เป็น `AWARDED`; เมื่อปฏิเสธ RFQ กลับ `OPEN` และลบแถว `award` (ประวัติอยู่ใน `audit_log`)

ตารางที่ 4-8 ลำดับการทำงานของการเทียบราคาและเสนอผล

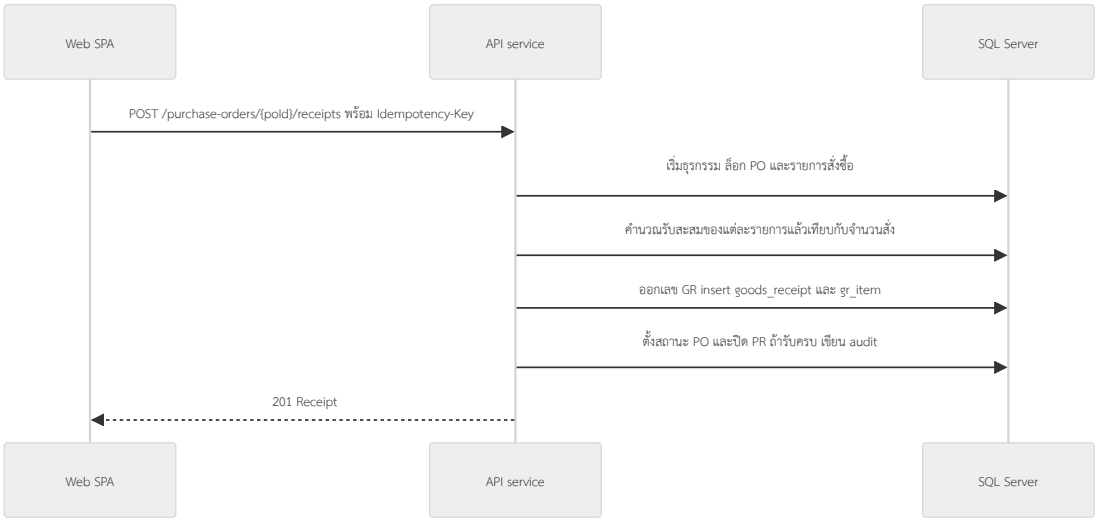


รูปที่ 4-8 ลำดับการสร้าง อ่าน และออกใบสั่งซื้อ

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
1	`createPurchaseOrder`	โหมดซื้อตรง (ไม่ส่ง `rfqId`): PR ต้องมีอยู่ (ไม่พบ `422 ERR-VAL-001` ที่ `prId`) และเป็น `APPROVED` และวงเงินไม่เกิน 5,000 บาท (BR-01) ต้องระบุ `vendorId`	PR ยังไม่อนุมัติ `409 ERR-STATE-002`; เกินวงเงิน `422 ERR-PO-001`
2	`createPurchaseOrder`	โหมดจากผลประกวด (ส่ง `rfqId`): RFQ ต้องเป็น `AWARDED` และเป็นของ PR นี้ ผู้ขายคือผู้ชนะ ไม่รับ `vendorId` จาก client	RFQ ยังไม่ `AWARDED` `409 ERR-STATE-001`
3	`createPurchaseOrder`	ผู้ขายต้อง `ACTIVE` (BR-11)	`422 ERR-VND-002`
4	`createPurchaseOrder`	รายการสั่งซื้อ: ซื้อตรงใช้จำนวนและราคาประมาณการของ PR; จากผลประกวดใช้จำนวนของ PR และราคาต่อหน่วยจาก `quotation_item` ของใบเสนอราคาที่ชนะ มูลค่ารวม = ผลรวมของ ปัดเศษ 2 ตำแหน่ง (จำนวน × ราคา) ซื้อตรงต้องไม่เกินวงเงิน PR (BR-09)	`po_item` ผูกกับ `pr_item`

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
5	`createPurchaseOrder`	ออกเลข `PO-YYYY-NNNNNN` (NFR-04) insert `purchase_order` สถานะ `DRAFT` ย้าย PR เป็น `ORDERED` เขียน audit	`201`
6	`getPurchaseOrder`	อ่าน PO พร้อมรายการ ชื่อผู้ขาย รับสะสมต่อรายการ (ผลรวม `qty_received` จาก `gr_item`) และคงเหลือ = สั่ง ลบ รับสะสม	`404 ERR-NF-001` เมื่อไม่พบ
7	`issuePurchaseOrder`	PO ต้องเป็น `DRAFT` ผู้ขายต้องยัง `ACTIVE` ตั้ง `ISSUED` และ `issued_at` เขียน audit (`ISSUE`)	สถานะอื่น `409 ERR-STATE-001`; ผู้ขายถูกระงับ `422 ERR-VND-002`

ตารางที่ 4-9 ลำดับการทำงานของใบสั่งซื้อ

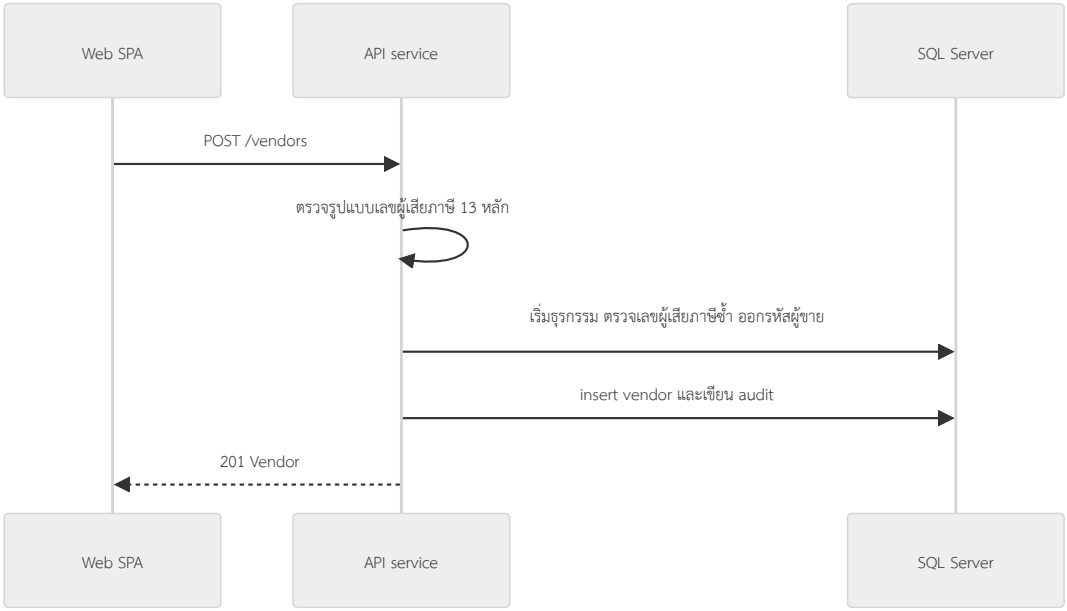


รูปที่ 4-9 ลำดับการบันทึกรับของ

ลำดับ	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
1	PO ต้องเป็น `ISSUED` หรือ `PARTIALLY_RECEIVED`	อื่น `409 ERR-STATE-001`
2	ทุก `poItemId` ต้องเป็นรายการของ PO นี้ ไม่ซ้ำ และมีอย่างน้อยหนึ่งรายการที่ `qtyReceived` มากกว่า 0	`422 ERR-VAL-001`
3	ล็อกแถว PO ไว้จนจบธุรกรรม เพื่อไม่ให้การรับของสองรายการพร้อมกันนับรับสะสมซ้ำ	การรับพร้อมกันรายการที่สองรอคิว
4	รับสะสมใหม่ = ผลรวม `qty_received` เดิมของรายการ + `qtyReceived` ที่ส่งมา ต้องไม่เกินจำนวนสั่ง จำนวนที่ปฏิเสธ (`qtyRejected`) ไม่นับเป็นรับ (BR-10)	เกิน `422 ERR-GR-001` ทั้งใบ ไม่บันทึกบางรายการ

ลำดับ	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
5	ออกเลข `GR-YYYY-NNNNN` (NFR-04) insert `goods_receipt` (`received_by` คือผู้เรียก) และ `gr_item`	`201` พร้อมเลขที่ใบรับของ
6	ทุกรายการรับสะสมเท่ากับจำนวนสั่ง: PO เป็น `RECEIVED` และ PR เป็น `CLOSED` ; นอกนั้น PO เป็น `PARTIALLY_RECEIVED`	`poStatus` ใน response เป็นสถานะใหม่
7	เขียน `audit_log` (`RECEIVE`) ในธุรกรรมเดียวกัน	

ตารางที่ 4-10 ลำดับการทำงานของการรับของ

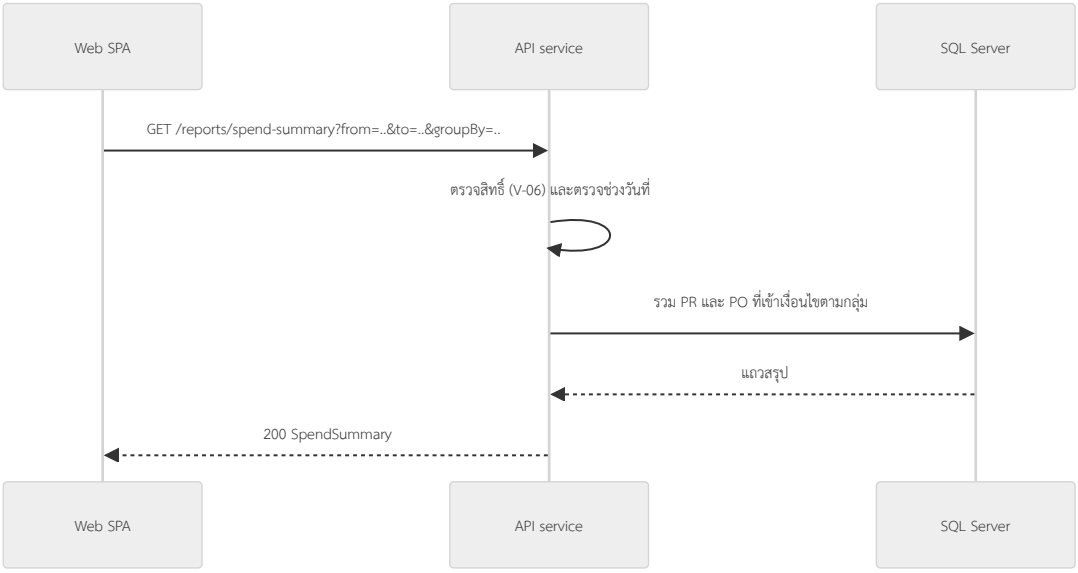


รูปที่ 4-10 ลำดับการจัดการทะเบียนผู้ขาย

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
1	`listVendors`	ค้น `q` กับชื่อ รหัสผู้ขาย และเลขผู้เสียภาษี (ตรงบางส่วน) กรองด้วย `status` เรียงตามรหัส	ไม่มีผลตรงตอบ `200` รายการว่าง
2	`createVendor`	`taxId` ต้องเป็นตัวเลข 13 หลัก	ผิดรูปแบบ `422` `ERR-VAL-001`
3	`createVendor`	`taxId` ต้องไม่ซ้ำ (ดัชนีเฉพาะที่ฐานข้อมูลเป็นตัวตัดสินสุดท้ายเมื่อมีการสร้างพร้อมกัน)	ซ้ำ `409` `ERR-VND-001`
4	`createVendor`	ออกรหัส `V` ตามด้วยเลข 4 หลักตาม `vendor_id` (ไม่ใช้รหัสซ้ำ) ในธุรกรรมเดียวกับการ insert สถานะเริ่มต้น `ACTIVE` เขียน audit; รองรับ 9,999 ราย เกินนั้นต้องขยายรูปแบบรหัสก่อน	`201`

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
5	`setVendorStatus`	เฉพาะ `PROCUREMENT_HEAD` เปลี่ยน `status` เป็น `ACTIVE` หรือ `SUSPENDED` เขียน audit (`entity` เป็น `VENDOR`) โดยเก็บ `reason` ใน `detail`; `reason` บังคับเมื่อตั้ง `SUSPENDED`	ไม่พบผู้ขาย `404 ERR-NF-001`; ไม่มี `reason` หรือ `status` ผิด `422 ERR-VAL-001`
6	`setVendorStatus`	การระงับมีผลกับการเชิญและการออก PO ครั้งต่อไปเท่านั้น (BR-11) RFQ และ PO ที่ออกไปแล้วไม่ถูกเปลี่ยน	

ตารางที่ 4-11 ลำดับการทำงานของทะเบียนผู้ขาย



รูปที่ 4-11 ลำดับการสร้างรายงานสรุปการจัดซื้อและการอ่าน audit trail

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
1	`getSpendSummary`	`from` ต้องไม่เกิน `to`	ผิด `422 ERR-VAL-001`
2	`getSpendSummary`	PR ที่นับ: `created_at` (แปลงเป็นปฏิทิน Asia/Bangkok) อยู่ในช่วง `from`–`to` รวมวันสุดท้าย สถานะไม่ใช่ `DRAFT` หรือ `CANCELLED` และมี PO ที่ไม่ใช่ `DRAFT` หรือ `CANCELLED` อย่างน้อยหนึ่งใบ	PR ที่ยังไม่มี PO ไม่อยู่ในรายงาน
3	`getSpendSummary`	กลุ่มตาม `groupBy` (ไม่ส่งใช้ `department`): `department` คือหน่วยงานของ PR `vendor` คือผู้ขายของ PO `month` คือเดือน `YYYY-MM` ของ `created_at` ของ PR	เรียงตามชื่อกลุ่ม

ลำดับ	operation	การทำงานภายใน	ผลลัพธ์หรือข้อผิดพลาด
4	`getSpendSummary`	`prCount` = จำนวน PR ไม่ซ้ำในกลุ่ม; `poCount` = จำนวน PO; `estimatedTotal` = ผลรวมวงเงินประมาณการของ PR ไม่ซ้ำในกลุ่ม (นับ PR ละหนึ่งครั้ง); `orderedTotal` = ผลรวมมูลค่า PO; `savingVsEstimate` = `estimatedTotal` ลบ `orderedTotal`	ค่าติดลบได้เมื่อสั่งเกินประมาณการ
5	`listAuditLogs`	เฉพาะ `ADMIN` อ่าน `audit_log` ตาม `entity` และ `entityId` เรียงตามเวลาจากเก่าไปใหม่ ตารางนี้รับเฉพาะ INSERT (NFR-02)	ไม่มีเหตุการณ์ตอบรายการว่าง

ตารางที่ 4-12 สูตรและลำดับการทำงานของรายงานและ audit trail

5. โครงสร้างโค้ดของ API service (Development View)

5.1 DEV-01 หน่วยโค้ดของ API service

API service (CTN-03) เป็น modular monolith โมดูลธุรกิจแต่ละตัว (CMP-01 ถึง CMP-07) แบ่งโค้ดเป็นสี่ชั้นเหมือนกัน และ CMP-08 เป็นโมดูลแพลตฟอร์มที่ทุกโมดูลใช้ร่วม

ชั้น	หน้าที่	ต้องไม่ทำ
Controller	รับ HTTP แปลง path query body เป็นคำสั่งของโมดูล ส่งผลกลับตามสัญญา OpenAPI แปลงข้อผิดพลาดของโมดูลเป็น `Problem`	ใส่กฎธุรกิจ เรียกฐานข้อมูลตรง
Application service	หนึ่ง operation หนึ่งเมธอด เปิดและปิดธุรกรรม เรียก policy และ repository ตามลำดับใน runtime view ของ operation นั้น เขียน audit	รู้จัก HTTP หรือรูปแบบ JSON
Domain policy	กฎล้วนๆ ไม่มี I/O: สายอนุมัติตามวงเงิน แยกหน้าที่ การเปลี่ยนสถานะที่อนุญาต การคำนวณวงเงินและการปัดเศษ เกณฑ์ซื้อตรง รับของไม่เกินจำนวนสั่ง	เรียก repository หรืออ่านเวลาปัจจุบันเอง (รับเวลาเป็นพารามิเตอร์)
Repository	คำสั่ง SQL ของตารางที่โมดูลเป็นเจ้าของ ส่งค่า `rowVersion` และจัดการ `If-Match`	ใส่กฎธุรกิจ เปิดธุรกรรมเอง

ตารางที่ 5.1-1 ชั้นของโค้ดในแต่ละโมดูล

กฎ	ความหมาย
ชั้นพึ่งพาลงล่างเท่านั้น	Controller → Application service → Domain policy และ Repository; Domain policy ไม่พึ่งชั้นอื่น
โมดูลไม่อ่านตารางของโมดูลอื่น	ต้องเรียก application service ของโมดูลเจ้าของ เช่น การสร้าง RFQ อ่านและเปลี่ยนสถานะ PR ผ่าน CMP-01

กฎ	ความหมาย
ธุรกรรมเป็นของ application service	เมธอดของ operation หนึ่งตัวเปิดธุรกรรมเดียว การเรียกข้ามโมดูลอยู่ในธุรกรรมเดียวกัน
CMP-08 ให้บริการร่วม	ตรวจ token และบทบาท (V-06) ออกเลขเอกสาร (`usp_next_doc_no`) เขียน audit ตรวจ `Idempotency-Key` แปลงข้อผิดพลาดเป็น `Problem`
สัญญา OpenAPI เป็นต้นฉบับ	ชนิดของ request และ response สร้างจาก `api/openapi.yaml` ไม่เขียนมือซ้ำ

ตารางที่ 5.1-2 ทิศทางการพึ่งพาของโมดูล

5.2 DEV-02 โมดูลกับตารางและ operation

โมดูล	ตารางที่เป็นเจ้าของ	เรียกโมดูลอื่น
CMP-01 ใบขอซื้อ	`purchase_request`, `pr_item`	CMP-08 (เลขเอกสาร audit)
CMP-02 อนุมัติ	`approval`	CMP-01 และ CMP-03 (เปลี่ยนสถานะเอกสารเมื่ออนุมัติครบ) CMP-08
CMP-03 จัดหา	`rfq`, `rfq_vendor`, `quotation`, `quotation_item`, `award`	CMP-01 (สถานะ PR) CMP-02 (สร้างสายอนุมัติผล) CMP-05 (ตรวจผู้ขาย) CMP-08
CMP-04 สั่งซื้อและรับของ	`purchase_order`, `po_item`, `goods_receipt`, `gr_item`	CMP-01 CMP-03 CMP-05 CMP-08
CMP-05 ผู้ขาย	`vendor`	CMP-08
CMP-06 รายงาน	ไม่มี (อ่านอย่างเดียว)	อ่านตารางของโมดูลอื่นด้วยคำสั่งรายงานเฉพาะ
CMP-07 audit	`audit_log`	ถูกเรียกผ่าน CMP-08
CMP-08 แพลตฟอร์ม	`app_user`, `department`, `doc_sequence`	ไม่เรียกโมดูลธุรกิจ

ตารางที่ 5.2-1 โมดูลของ API service กับตารางที่เป็นเจ้าของ

5.3 DEV-03 การสร้างและรันบนเครื่องนักพัฒนา

ชุดอ้างอิง (`reference/`) สร้างตาม ADR-01 และรันด้วย `docker compose up -d --build` ในโฟลเดอร์นั้น ได้ปลายทาง `http://localhost:8080/api/v1` ผ่าน reverse proxy ชุดทดสอบ API (`tests/api-trial`) ยิงที่ปลายทางนี้ ตารางนี้รวมข้อจำกัดของ stack ที่เจอตอนรันจริง ซึ่งเอกสารส่วนอื่นไม่ได้บอก

ส่วนประกอบ	ภาพ	หน้าที่และข้อควรรู้
ฐานข้อมูล	SQL Server 2022	สร้างจาก `sql/schema.sql` และ `sql/seed-data.sql` โดยงานครั้งเดียวที่รันก่อน API; บนเครื่อง Apple silicon ต้องรันภาพ `linux/amd64` ผ่านการจำลอง

ส่วนประกอบ	ภาพ	หน้าที่และข้อควรรู้
บัญชีแอป	`app`	อ่านเขียนข้อมูลและรันขั้นตอนที่เก็บไว้ได้ แต่แก้หรือลบ `audit_log` ไม่ได้ (schema สั่ง DENY ให้ PUBLIC) API ต้องเชื่อมด้วยบัญชีนี้ ไม่ใช่ `sa`
แคช	Redis 7	เก็บผลของ `Idempotency-Key` 24 ชั่วโมง
API	ASP.NET Core 8	อ่านค่าตั้งต้นจากตัวแปรสภาพแวดล้อม; เปิดโปรไฟล์ทดสอบด้วย `APP_PROFILE=test` ห้ามเปิดในสภาพแวดล้อมจริง
reverse proxy	nginx	พอร์ตเดียวที่เปิดออก ส่ง `/api/` ต่อให้ API

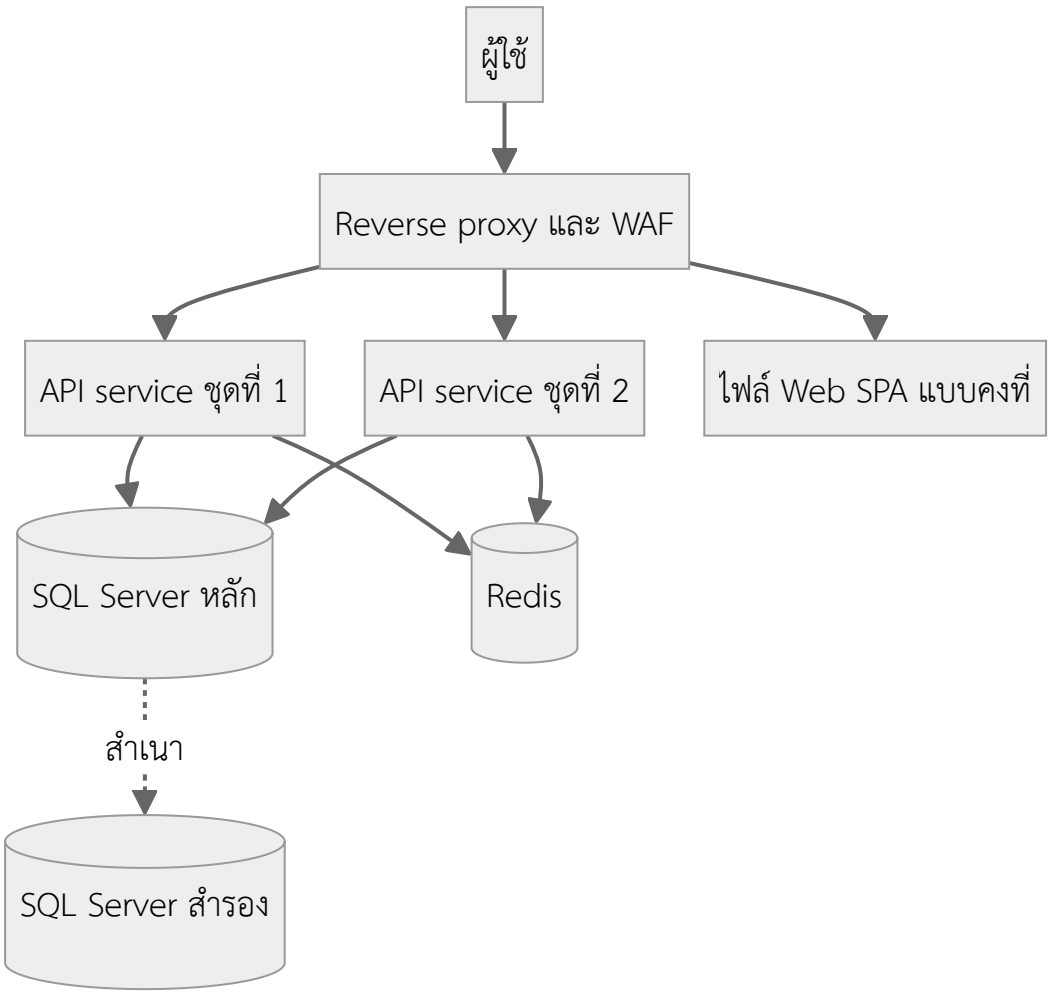
ตารางที่ 5.3-1 ส่วนประกอบของชุดรันบนเครื่องนักพัฒนา

เรื่อง	ข้อกำหนด
ไลบรารีเชื่อม SQL Server	ต้องมี ICU ในภาพ ห้ามเปิดโหมด invariant globalization ไม่เช่นนั้นเปิดการเชื่อมต่อไม่ได้
ชื่อคอลัมน์ผลลัพธ์	`LINENO` เป็นคำสงวนของ T-SQL ฟิลด์ `lineNo` ของ API ต้องตั้งเป็นชื่อในวงเล็บเหลี่ยมเมื่อใช้เป็นชื่อแทนคอลัมน์
เลขที่เอกสาร	เรียก `usp_next_doc_no` นอกธุรกรรมธุรกิจ (ดู V-08 ขั้น 2)
ข้อผิดพลาดจากดัชนีเฉพาะ	ตัวตัดสินสุดท้ายเมื่อมีคำขอพร้อมกัน; ต้องแปลงเป็นรหัสข้อผิดพลาดของ API ตามที่ runtime view ระบุ (เช่น `taxId` ซ้ำเป็น `ERR-VND-001`) ไม่ปล่อยเป็น `500`
เวลา	เก็บ `DATETIME2(0)` เป็น UTC ละเอียดระดับวินาที เวลาที่ใส่ในแถวมาจากนาฬิกาของคำขอ (ควบคุมได้ด้วย `X-Test-Now` ในโปรไฟล์ทดสอบ) ไม่ใช้ค่าเริ่มต้นของฐานข้อมูล

ตารางที่ 5.3-2 ข้อจำกัดของ stack ที่ต้องรู้ก่อนเขียนโค้ด

6. การติดตั้ง

6.1 แผนภาพการติดตั้ง



รูปที่ 6.1-1 มุมมองการติดตั้งในสภาพแวดล้อมใช้งานจริง

6.2 สภาพแวดล้อม

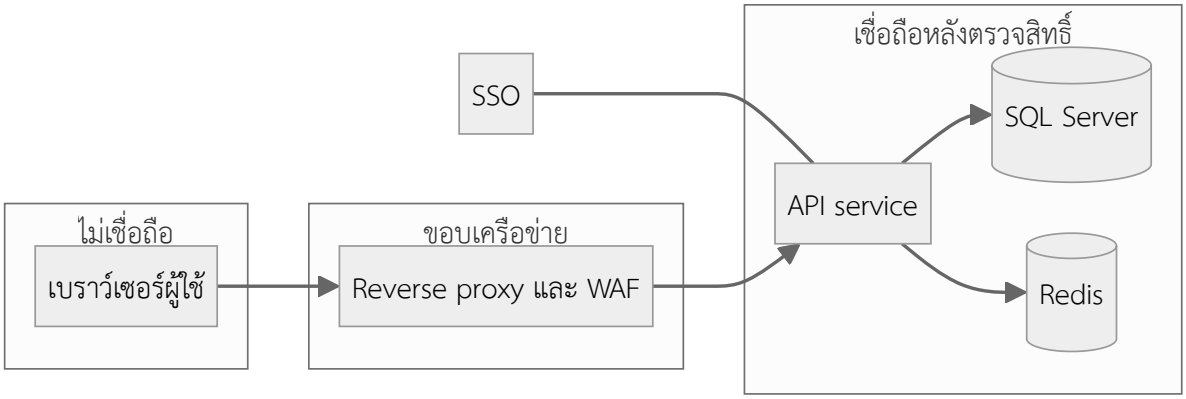
สภาพแวดล้อม	วัตถุประสงค์	ข้อมูล	ผู้ใช้งาน
Dev	พัฒนาและทดสอบหน่วยโดยนักพัฒนา	ข้อมูลสมมติ	ทีมพัฒนา
SIT	QA ทดสอบระบบรวมโดยใช้เกณฑ์ตรวจรับเป็นฐาน	ข้อมูลสมมติที่สร้างจากข้อมูลตัวอย่าง	ทีมพัฒนาและ QA
UAT	ผู้ใช้ตรวจรับตาม Journey	ชุดข้อมูลที่ผู้ใช้เตรียม ไม่มีข้อมูลจริงที่เป็นความลับ	ผู้ใช้และ QA

สภาพแวดล้อม	วัตถุประสงค์	ข้อมูล	ผู้ใช้งาน
Production	ใช้งานจริง	ข้อมูลจริง	ผู้ใช้ทั้งหมด ผู้ดูแลระบบจัดการ

ตารางที่ 6.2-1 สภาพแวดล้อมของระบบ

7. ความมั่นคงปลอดภัย

7.1 ขอบเขตความเชื่อถือ



รูปที่ 7.1-1 ขอบเขตความเชื่อถือ

7.2 มาตรการหลัก

7.2.1 การพิสูจน์ตัวตน

ผู้ใช้ยืนยันตัวตนกับ SSO ด้วย OpenID Connect และส่ง JWT มากับทุกคำขอ API ตรวจลายเซ็น ผู้ออกโทเค็น และวันหมดอายุทุกครั้ง โทเค็นที่หมดอายุหรือไม่ถูกต้องตอบ `ERR-AUTH-001` รหัสผู้ใช้และบทบาทอ่านจาก claim ไม่รับจากเนื้อหาคำขอ

7.2.2 การให้สิทธิ์

ตรวจสองชั้น: บทบาทต้องอยู่ใน `x-roles` ของ operation แล้วตรวจขอบเขตข้อมูลตามหน่วยงานและเจ้าของเอกสารใน service ไม่ผ่านตอบ `ERR-AUTH-002` กฎแยกหน้าที่ (ห้ามอนุมัติของตนเอง) ตรวจที่โมดูลอนุมัติและตอบ `ERR-SOD-001` การซ่อนปุ่มที่หน้าจอเป็นเพียงความสะดวกของผู้ใช้ ไม่ใช่มาตรการความปลอดภัย

7.2.3 audit trail

ทุกการเปลี่ยนสถานะและการตัดสินใจเขียนลง `audit_log` ในธุรกรรมเดียวกับการเปลี่ยนแปลง ตารางนี้เพิ่มอย่างเดียว บัญชีที่แอปใช้ไม่มีสิทธิ์ `UPDATE` หรือ `DELETE` ตรวจแล้วด้วยชุดตรวจสอบ schema (error 229)

7.3 การเทียบกับ OWASP Top 10

ความเสี่ยง	มาตรการ	เกณฑ์ตรวจรับที่รองรับ
A01 การควบคุมการเข้าถึงที่บกพร่อง	บทบาทและขอบเขตข้อมูลตรวจที่ API, แยกหน้าที่ในโมดูลอนุมัติ	AC-02-01, AC-04-01, AC-04-05, AC-12-02, AC-02-04
A03 การโจมตีแบบ injection	คำสั่งฐานข้อมูลใช้พารามิเตอร์ ไม่ต่อสตริง และตรวจชนิดและช่วงค่าของข้อมูลเข้า	AC-01-02, AC-01-03, AC-04-02, AC-11-02
A04 การออกแบบที่ไม่ปลอดภัย	เครื่องสถานะของเอกสารและ Idempotency-Key กันการทำซ้ำและการข้ามขั้น	AC-03-05, AC-03-06, AC-09-05
A07 การพิสูจน์ตัวตนบกพร่อง	ใช้ SSO ไม่เก็บรหัสผ่าน ตรวจ token ทุกคำขอ	AC-02-02
A08 ความสมบูรณ์ของข้อมูลและซอฟต์แวร์	audit แก้ไม่ได้ระดับฐานข้อมูล	AC-13-02
A09 การบันทึกและเฝ้าระวังบกพร่อง	ทุกการเปลี่ยนสถานะมี audit ในธุรกรรมเดียวกัน	AC-13-01

ตารางที่ 7.3-1 ความเสี่ยงตาม OWASP และมาตรการ

8. คุณลักษณะคุณภาพและมาตรการเชิงสถาปัตยกรรม

NFR	ข้อกำหนด	มาตรการ	ตรวจด้วย
NFR-01	หน้ารายการ PR ตอบเร็วเมื่อมี PR 100,000 รายการ	keyset pagination บนดัชนีครอบคลุม `ix_pr_status_created` ไม่ใช้ offset	
NFR-02	audit trail แก้ไม่ได้	ตาราง `audit_log` เพิ่มอย่างเดียว และ `DENY UPDATE, DELETE` ให้บัญชีแอปตรวจที่ระดับฐานข้อมูล	AC-13-02
NFR-03	ทุก endpoint ตรวจสิทธิ์ตามบทบาทและขอบเขตข้อมูล	JWT จาก SSO, ตรวจ `x-roles` ต่อ operation, กรองข้อมูลตามหน่วยงานใน service, แยกหน้าที่ในโมดูลอนุมัติ	AC-02-01, AC-04-01, AC-04-05, AC-12-02, AC-02-04
NFR-04	เลขเอกสารไม่ซ้ำแม้ออกพร้อมกัน	ขั้นตอน `usp_next_doc_no` ล็อกแถวเดียวต่อ (ชนิด, ปี) ไม่ใช้ `MAX()+1`	AC-14-01
NFR-05	ส่งคำสั่งซ้ำไม่ทำงานซ้ำ	แคชเก็บผลตาม `Idempotency-Key` 24 ชั่วโมง และกฎสถานะของเอกสารเป็นด่านที่สอง	AC-03-05, AC-03-06, AC-09-05
NFR-06	แสดงผลภาษาไทยและวันที่ พ.ศ.	จัดเก็บเวลา UTC ใน `DATETIME2` และแปลงเป็น พ.ศ. ที่ Web SPA ข้อความผิดพลาดเป็นภาษาไทยจาก API	

ตารางที่ 8-1 คุณลักษณะคุณภาพและมาตรการ

9. การตัดสินใจเชิงสถาปัตยกรรม

9.1 ADR-01 เทคโนโลยีหลัก (proposed)

บริบท: ต้องเลือกภาษาและฐานข้อมูลให้เข้ากับ schema T-SQL ที่ออกแบบไว้ (ROWVERSION, ขั้นตอนออกเลข, ดัชนีแบบมีเงื่อนไข)

การตัดสินใจ: ใช้ ASP.NET Core (.NET 8) ฝั่ง API, React และ TypeScript ฝั่งหน้าจอ และ SQL Server 2022

ผลที่ตามมา: ใช้ฟีเจอร์ของ SQL Server ได้เต็ม แต่ผูกกับผู้ขายฐานข้อมูลรายเดียว ทางเลือกอื่นที่ไม่ได้เลือกคือ Node.js หรือ Java กับ PostgreSQL ซึ่งต้องออกแบบ schema ใหม่

9.2 ADR-02 โครงสร้าง API (proposed)

บริบท: ขนาดโจทย์เล็ก ธุรกรรมหลายตารางต้อง commit พร้อมกัน (เช่น อนุมัติขั้นสุดท้ายเปลี่ยนทั้ง approval, PR และ audit)

การตัดสินใจ: ทำ API เป็น modular monolith ที่แยกโมดูลตามหน้าที่แต่ใช้ฐานข้อมูลเดียวและธุรกรรมเดียว

ผลที่ตามมา: ธุรกรรมง่ายและทดสอบง่าย ไม่ต้องจัดการข้อมูลสอดคล้องข้ามบริการ แยกเป็น microservices ภายหลังได้ยากขึ้นถ้าโมดูลอ้างตารางของกันโดยตรง จึงให้โมดูลคุยกันผ่านอินเทอร์เฟซ

9.3 ADR-03 endpoint อนุมัติกลาง (accepted)

บริบท: การอนุมัติ PR และผลเลือกผู้ชนะมีกฎร่วมกัน (สายอนุมัติ, แยกหน้าที่, บันทึกผล)

การตัดสินใจ: ใช้ `/approvals/{id}/decision` เดียว แยกชนิดงานด้วย `docType` แทน endpoint แยกต่อเอกสาร

ผลที่ตามมา: กฎอยู่ที่เดียว เพิ่มชนิดงานใหม่ไม่ต้องเพิ่ม endpoint แต่ `approval` ไม่มี foreign key ไปเอกสารต้นทาง ต้องบังคับด้วยตรรกะและ unique constraint

9.4 ADR-04 ที่เก็บ Idempotency-Key (proposed)

บริบท: ต้องเก็บผลของคำสั่ง 24 ชั่วโมง แต่ไม่ต้องการเพิ่มตารางในฐานข้อมูลหลัก

การตัดสินใจ: เก็บใน Redis พร้อม TTL ไม่เพิ่มตารางใน SQL Server

ผลที่ตามมา: เร็วและหมดอายุเอง แต่ถ้าแคชหาย การส่งซ้ำอาจทำงานซ้ำ จึงต้องมีกฎสถานะของเอกสารเป็นด่านที่สอง (`ERR-STATE-001`) และต้องทดสอบกรณีนี้

9.5 ADR-05 การยืนยันตัวตน (proposed)

บริบท: องค์กรมี SSO กลางอยู่แล้ว ระบบไม่ควรเก็บรหัสผ่าน

การตัดสินใจ: ใช้ OpenID Connect กับ SSO ขององค์กร รับ JWT ที่มีรหัสผู้ใช้และบทบาท และให้ API ตรวจลายเซ็นและวันหมดอายุทุกคำขอ

ผลที่ตามมา: ไม่ต้องจัดการรหัสผ่านและการลืมรหัส แต่ผูกกับความพร้อมของ SSO และต้องตกลงรูปแบบ claim ของบทบาทกับผู้ดูแล SSO ก่อนพัฒนา

9.6 ADR-06 API แบบไม่เก็บสถานะ (proposed)

บริบท: ต้องรันหลายชุดหลัง proxy ตามมุมมองการติดตั้ง

การตัดสินใจ: API ไม่เก็บ session ในหน่วยความจำ สถานะทั้งหมดอยู่ในฐานข้อมูลหรือแคช

ผลที่ตามมา: เพิ่มจำนวนชุดได้โดยไม่มีผลต่อผู้ใช้ แต่ทุกคำขอต้องตรวจ token ใหม่